

v Imp



Notification System



* Design a Notification System

↳ [Plug and Play Model] → modal which can be integrated in any Application with minimum code change.

↳ Highly Extendible

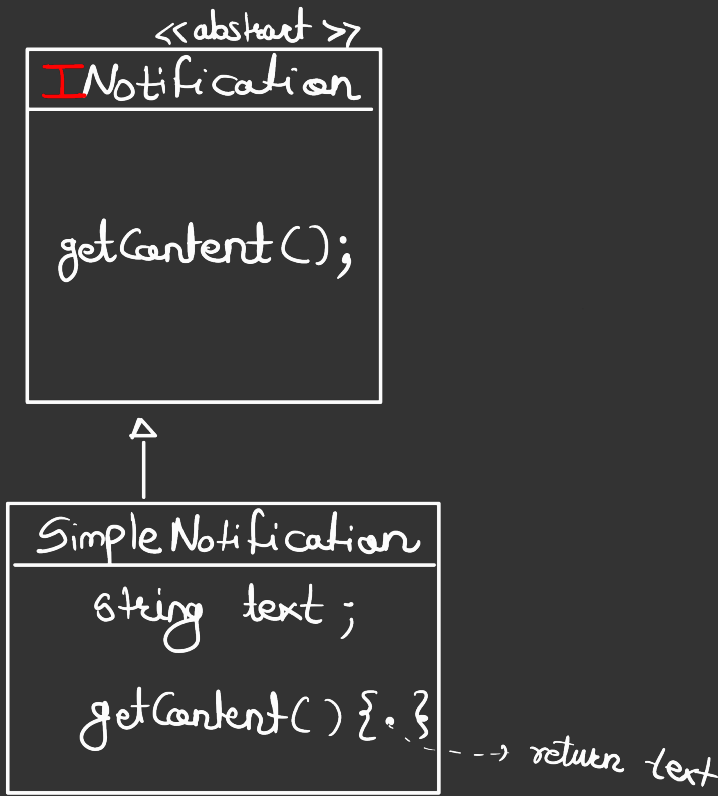
↳ : SMS, Email, Pop-up.
for now

↳ Extendible Notifications (dynamically)

↳ Store all Notification

↳ Logging all Notification

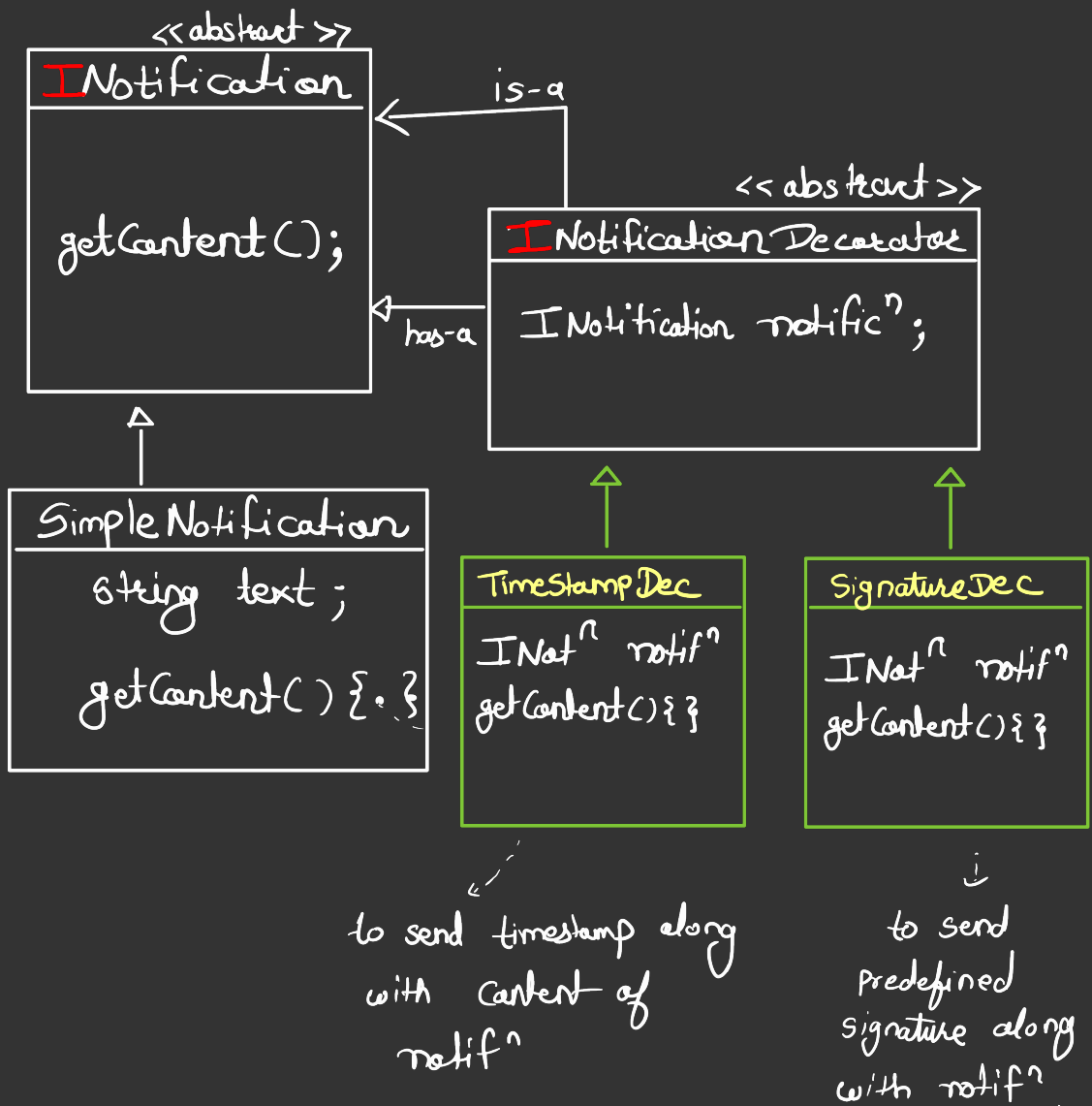
① Sabse pehle Notification class banani padegi.



Our requirement is extendible notification functionality dynamically.

↓
This can be achieved by Decorator Design Pattern

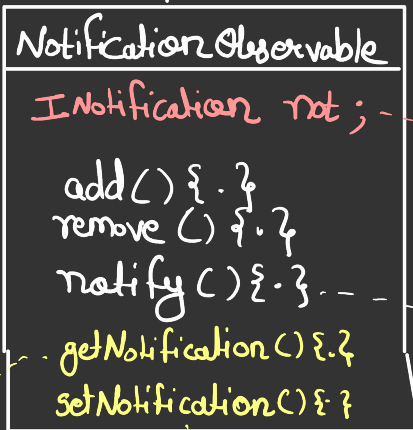
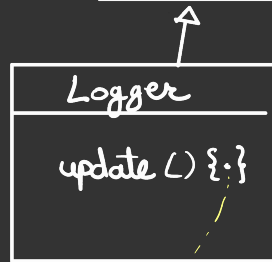
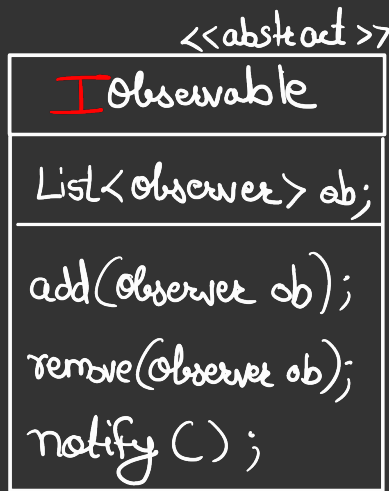
① So let's create Notification Decorator



② Now our requirement of extensible Notificⁿ dynamically is implemented.

⊙ Now, we have created Notification hierarchy,
Now something should be there to push the notifⁿ
so we will need NotificationService, This service
will push the notification to all the observer via
Observable.

⊙ So jo hume SMS, Email, pop-up pe hume notificⁿ
bhejna hai uska ek engine hoga, Notification
Engine jo decide karega SMS ko over the internet
kaise bhejte hai, Email ko over the internet kaise
bhejte hai Ye sab hum observer pe chor
denge, ek Observable hoga jo batayega
notification push hone wali hai.



internally calls `getNotification`.
then internally will
print an log file.

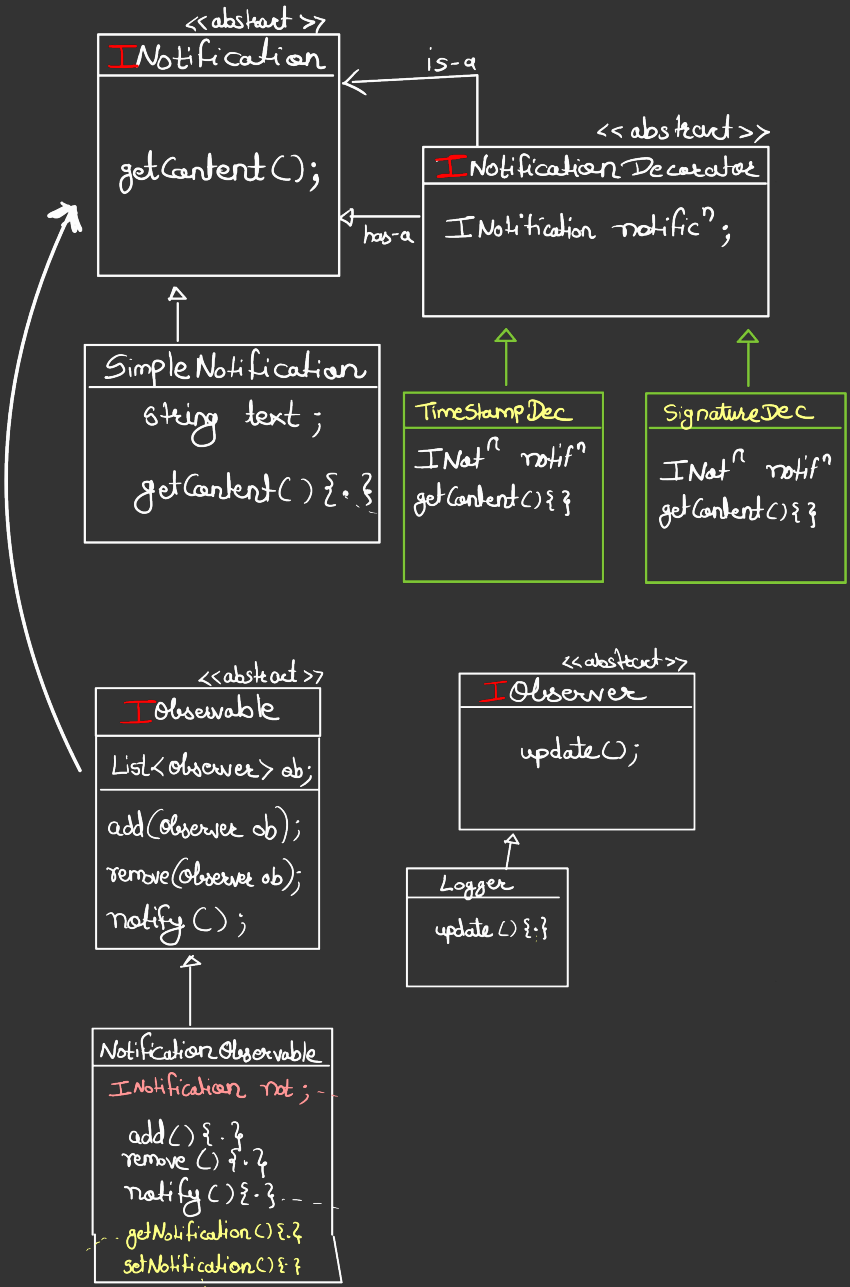
this means
`INotif` &
`NotificationObserver`
are related by
Has-a relation

observers ki list pe loop
karke uski `update()`
method call karega.

observer
is method
ko use
karke
notification
ka content
lega

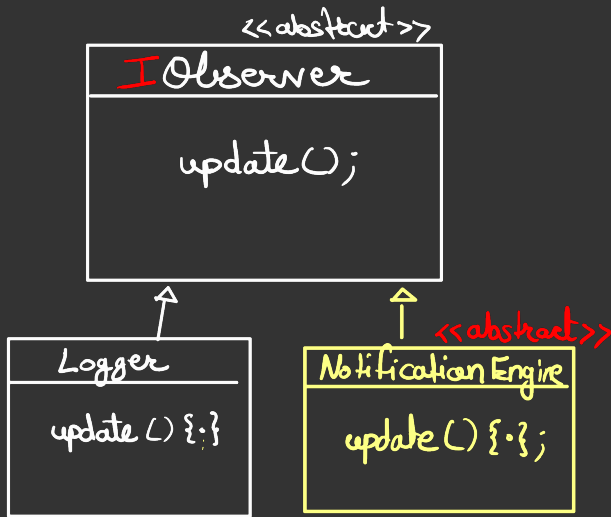
setter bhi
change
hoga
content
set
karne
ke liye

Design So Far

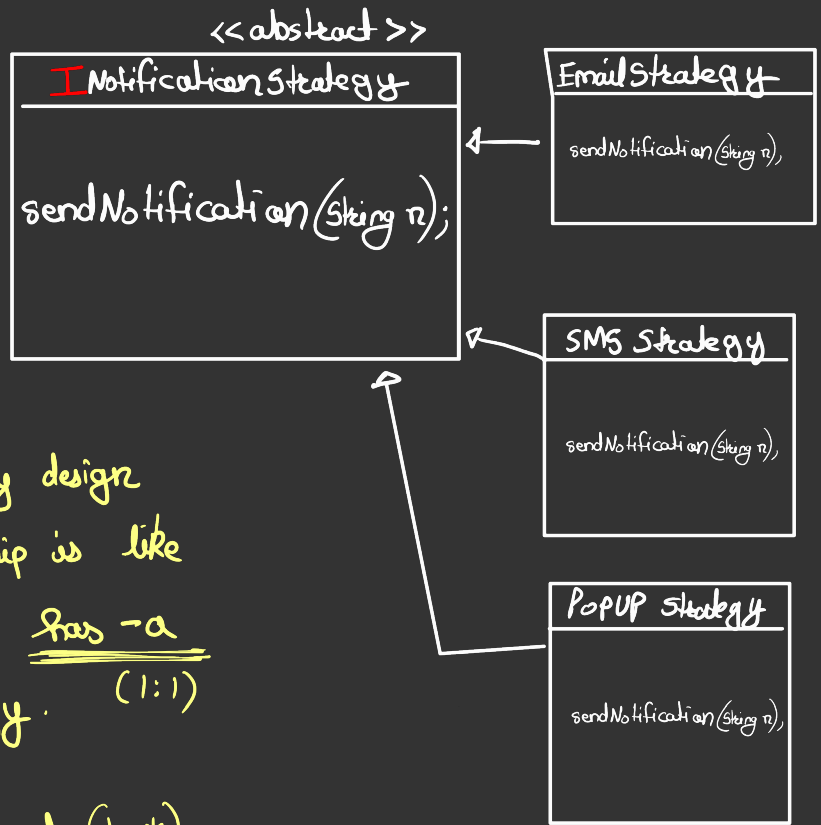


(#) Now, we want our notification to be sent over SMS, email, pop-up & should be extensible (OCP)

So let, keep one more `IObserver` called `Notification Engine`. This engine is responsible for sending notification of different types. Engine ko ye nahi pata hoga ki notification kaise bhejke hai, uske liye different strategies bana-enge.



⑧ Now let's create notification strategy,

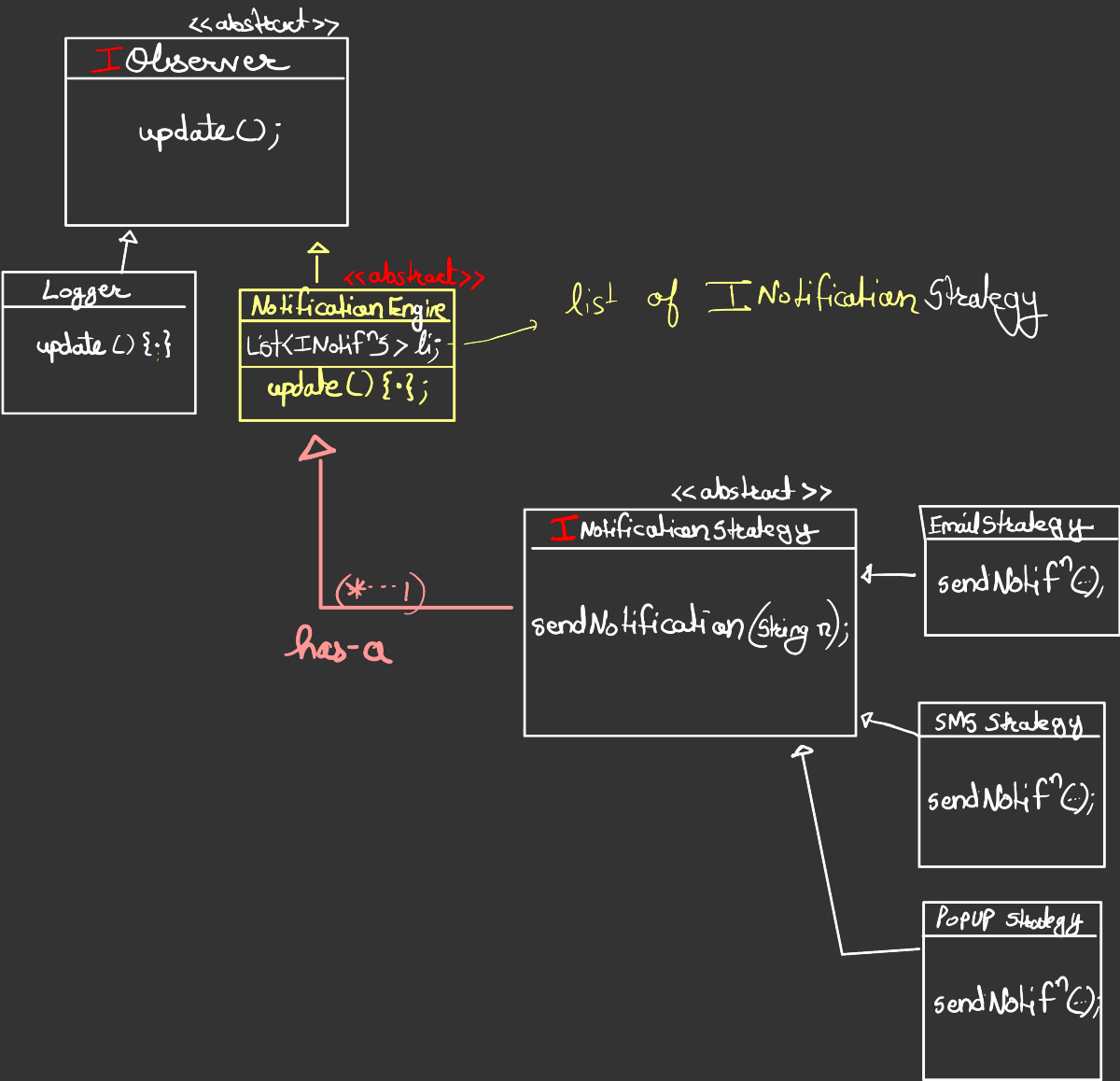


⑨ Generally in strategy design
Pattern, relationship is like
NotificationEngine has-a
NotificationStrategy. (1:1)

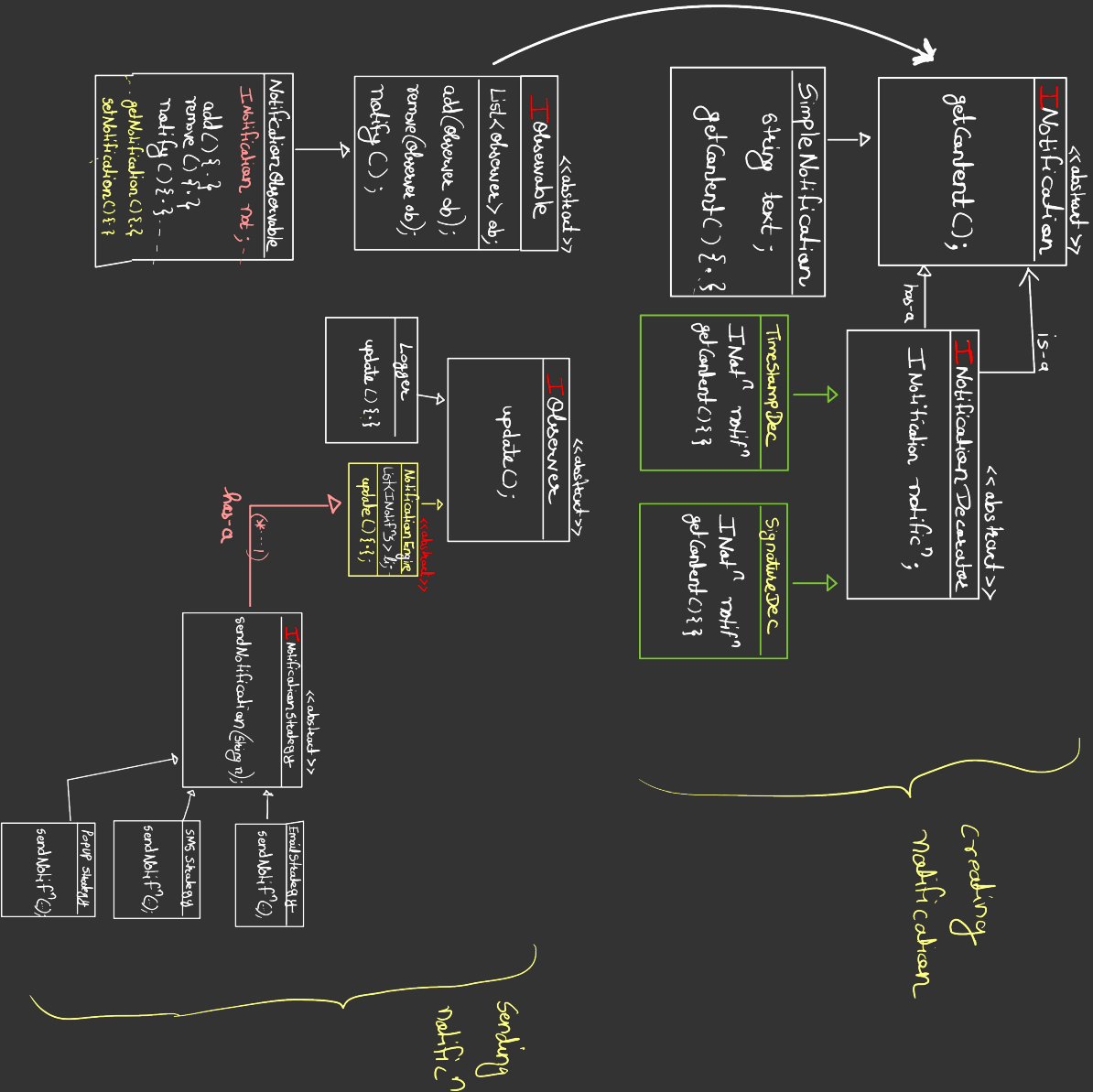
⑩ But here we need (1:*)
has-a relationship betⁿ
NotificationEngine & NotificationStrategy

slight change in Strategy Design Pattern.

ie :-> NotificationEngine has many NotificationStrategies



Design So far



⑧ Now only one thing is pending, that is we need a NotificationService which will attach the notification to Observable once it is created.

So, our NotificationService will sendNotification to Observable

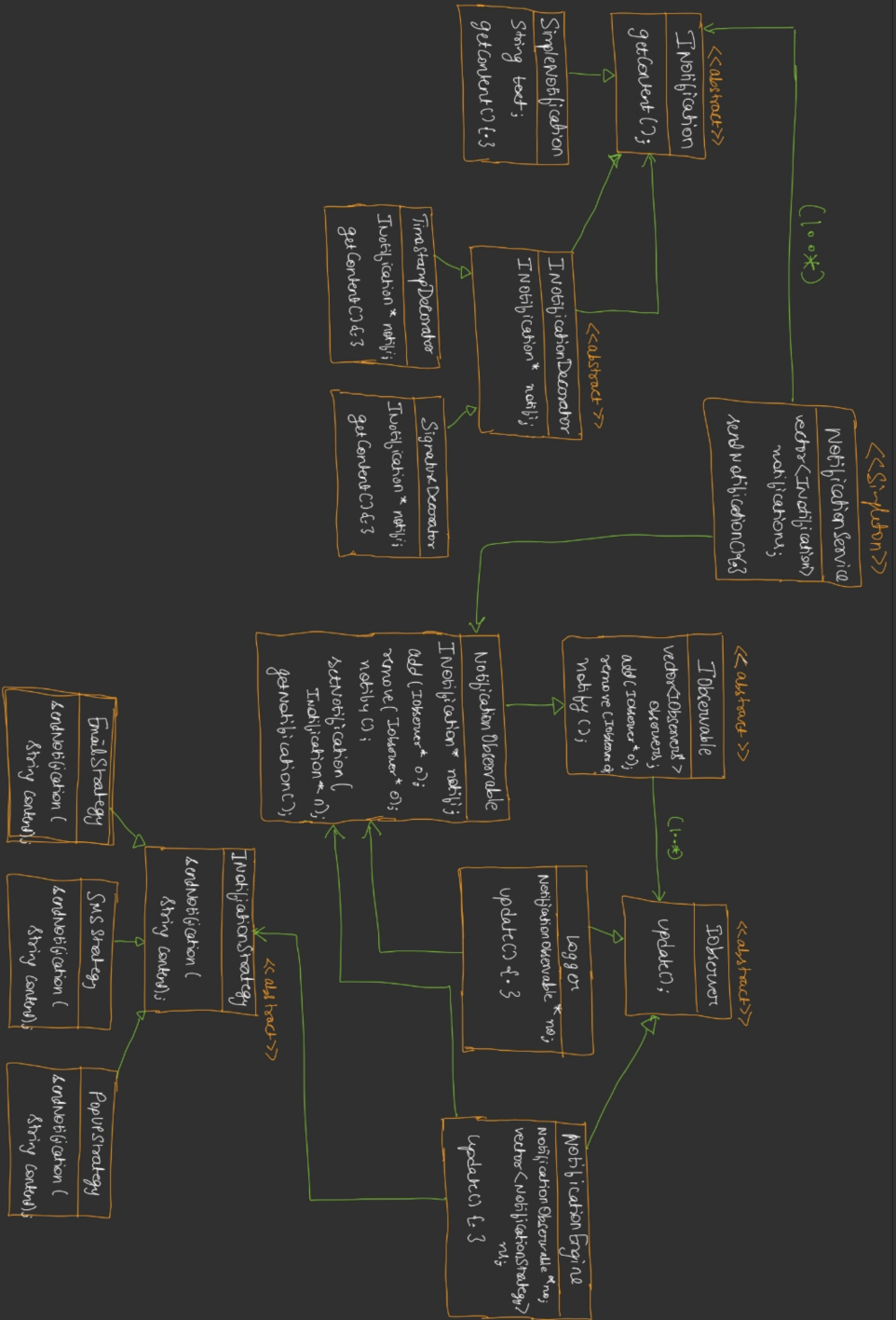
NotificationService
INotification notification;
IObservable ob;
sendNotification(notification){}

⑧ Also one of our requirement was to store all our notifications. So we can keep a list of notifications in NotificationService.

NotificationService
IObservable ob;
List<INotification> list;
sendNotification(notification){}

Final Design

Singleton



① All requirements Satisfied. ✓✓